claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_684a0618-ccb\agent-a050747f33fbd8a9d.jsonl`
- SHA-256: `eb026cc1ae3f63c21d00698f3fbc591df0c8943a6a977d6f5717ce3763e08480`
- Source modified: `2026-06-19T19:18:48+00:00`
- Imported at: `2026-07-08T16:00:03+00:00`
- project: `wf_684a0618-ccb`
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`

Transcript

1. user (2026-06-19T19:17:29.032Z)

THE IDEA TO ASSESS: a desktop app OR web UI where the user points KhelSutra at an ONLINE storage location they already own/pay for ? a Google Cloud Storage bucket, Google Drive, OneDrive/Microsoft 365, Dropbox, S3, etc. KhelSutra then WATCHES that folder, pulls/streams new recordings as they arrive, and AUTO-prepares highlights (same processing as the local/desktop path). "Bring your own storage + autosync": footage lands in their cloud ? reels appear automatically, zero manual upload.

You are assessing how this idea fits ONE area of the KhelSutra (badminton-highlight-indexer) docs. Read: docs/VIDEO_LOCALITY_MODEL.md. This is the "where the video lives" mental model. Does it already have a tier for "the user?s own cloud storage"? Does it discuss autosync / watch-folder? Map the idea onto its tiers (e.g. L0 fully-local ? cloud).

Be evidence-based and honest ? quote real anchors, don't invent. If the doc does NOT anticipate the idea, say so plainly and note what it WOULD take. Return the structured object.

2. assistant (2026-06-19T19:17:34.015Z)

I'll read the target doc to assess how the BYO-storage/autosync idea fits.

3. user (2026-06-19T19:17:34.170Z)

```
# Video locality ? where the bytes live (mental model + open questions)

**Status:** THINKING DOC (owner-requested, 2026-06-12) ? a shared mental model for
the upload/storage problem, with the open questions named. **No implementation is
greenlit by this doc** (per CLAUDE.md, both tracks stay owner-gated).
**Companions:** `UI_PROPOSAL.md` (B2/B3 upload/download), `HOSTING_PLAN.md`,
`PLATFORM_ARCHITECTURE.md` §3 (future StorageBackend/ComputeBackend registries),
collector ADR-002 (the frame-identical proxy contract this doc leans on).

## 0. The questions that triggered this (and the short answers)

**"Do we store the video they upload?"** Yes ? today the chunked-upload flow
assembles the full file under `security.upload_dir` (default `output/uploads`) on
our box, registers it by MD5, and nothing ever deletes it (retention policy is an
acknowledged gap ? BACKLOG). Processing also strews derived artifacts (chunks,
frames, trajectories, WSL temp) and, on the Gemini path, uploads ?1280px
re-encoded chunks to Google's Files API.

**"How would they upload a 10 GB file?"** Today they can't: `/api/upload/init`
rejects anything over `security.max_upload_mb` (default **4096 MB**), and even
under the cap the file moves as sequential ?90 MB parts through a free-tier edge
whose body limit forced that part size. For scale: 10 GB ? 30?45 min of 4K phone
video (or ~2?3 h at 1080p); on a typical Indian uplink that's **~1 h at 20 Mbps,
```

~4.5 h at 5 Mbps** ? before retries. Browser-tab uploads of that size are miserable everywhere; expecting academies to manage S3/GCS buckets is a non-starter. **The 10 GB problem is real and the current design doesn't solve it.**

"Can we make it so there is NO upload?" Mostly ? because of §1.

1. The central insight: timestamps are the product; the video is dead weight

Walk the pipeline and ask what each stage actually consumes:

- The **Gemini segmenter** never sees the original ? it re-encodes chunks down to **1280 px** (`indexing.ai_chunk_scale_width`) before uploading them to Google.
- The **detectors/featurizer** consume downscaled frames and trajectories.
- The only consumer of the original full-res bytes is the **highlight compiler**, and only to cut clips ? a pure-ffmpeg operation that can run **wherever the original lives**.
- What the customer receives is `{rally intervals, report, reel}`. Intervals + report are **kilobytes**. Given the intervals, the reel can be cut on the machine that already has the original.

So: **any design that moves the 10 GB is moving the wrong thing.** The question is never "how do we upload the video" but "what is the smallest artifact that must cross the network for this tier of service."

2. The locality ladder

From most to least bytes leaving the academy's Windows desktop. Sizes are rough ratios for a 4K source ? measure on a real file before committing to any tier.

	What crosses the network	~Bytes (10 GB source)	What it needs	Status
L3 ? full upload (today's flow)	everything	10 GB (blocked: 4 GB cap)	paid edge tier + resumable uploads + server disk + retention policy	built, capped, wrong for big files
L2 ? proxy upload	a timeline-identical ?1080p re-encode	~1?2.5 GB	a tiny client prep step (ffmpeg) ? **the exact frame-identical contract we already built and verified as collector `prep-annotation-proxy`** (same fps, same frame count, parity-checked, MD5?MD5 manifest)	contract proven; client wrapper + server acceptance = new work
L1 ? chunks/candidates upload	only the ?1280px AI chunks (or just candidate windows) the Gemini path needs anyway	~0.3?1 GB	client-side chunk+downscale (ffmpeg, CPU); server becomes a **GPU-less orchestrator**	the server half exists (it's the gemini segmenter's own flow, relocated)
L1-direct	chunks go **straight from the client to Gemini** ? zero video bytes through our server	~0 through us	per-user key custody / metering / cost ceilings (OQ4)	idea only
L0 ? fully local	nothing (calls to Google only if the Gemini path is used)	0	the owned on-device model (roadmap north star) or a local GPU + our stack	not alpha-feasible; the long-term answer
L4 ? sneakernet	a hard drive	0 over network	a person (the field associate / owner)	how the pilot actually works today

The honest "no upload" answer: *strictly* no bytes leave only at L0 with the owned model (and even L0 leaks ?1280px chunks to Google while the Gemini path is the brain ? say that plainly in consent language, OQ3). The pragmatic near-term answer is **L2 or L1: don't eliminate the upload, shrink it 5?30×** ? at which point the existing 90 MB-part flow, the 4 GB cap, and a coffee-break wait are all suddenly fine.

3. Second-order effects worth noticing

- **L1/L2 make the serving box GPU-less for the Gemini tier** ? ffmpeg + API orchestration only. Hosting cost collapses; the GPU box stays for WASB/owned-model work. (PLATFORM §3's ComputeBackend split falls out naturally.)
- **Storing the proxy instead of the original** is ~10× less disk AND a smaller privacy surface; the review UI (alpha screen A4) actually *prefers* streaming a

1080p proxy to seeking inside a 10 GB original.
- **The correction loop survives** at every tier ?L1: labels are intervals on a timeline, and the proxy timeline IS the original timeline by construction (the collector ADR-002 invariants: same fps, same frame count, parity-verified).
- **video_id keying:** the indexer keys by MD5 of the file it processed. At L2 that's the proxy's MD5; the proxy-manifest pattern (original_md5 ? proxy_md5) already exists ? but the product needs ONE registered identity rule (OQ2).

4. The frugality frame ? afford the bills by not burning GPU

The runway constraint (owner, 2026-06-12): **the bills stay personally affordable for a long time as long as we don't burn too much GPU.** The locality ladder maps directly onto that:

- **The Gemini serving path costs ~$0.30?0.60 per hour of video** in API calls (2026-06-10 quality table) and at L1/L2 needs **no GPU at all** ? a CPU box, ffmpeg, and an API key. At pilot scale (a handful of academies, a few sessions a week) that is single-digit dollars a month: affordable indefinitely.
- **GPU spend is the thing to guard.** WASB / owned-model inference and training stay on the owner's already-paid-for local 2070S; rent nothing until the owned model has a ship-gate-passing reason to be served. L2/L1 turn this from a discipline into a default: *the serving box has no GPU to burn.*
- **Storage follows the same logic:** keep proxies, not originals (~10× less disk); settle retention (OQ1) before volume arrives; the free edge tier stays adequate precisely because L2 keeps payloads under the existing caps.
- **Put an explicit cost ceiling per video** (ties into OQ4): a 7-hour tape or a runaway retry must not surprise-bill. The existing ops learnings (serial uploads, `ai_max_workers=1`, bounded exponential retry) already point this way.

5. The academy-visit dividend ? field work builds the product, not just the verdict

The C13 visits aren't only a market read; run deliberately, each visit deposits product assets:

- **Phase-2 recordings (consented, adults-only) ARE the target corpus.** The owned model's ship target (0.66-to-beat) lives on exactly the messy multi-court footage class these academies produce ? every show-back visit grows the training set the model is starved for, at zero marginal engineering cost.
- **The show-back is a live dress rehearsal of the product loop:** footage comes back on a drive (L4), we proxy + process + return a reel ? the same motion L2 will automate. Every friction the associate hits there (file handoff, proxy time, reel delivery) is a pre-paid product-backlog item; write it down.
- **Demo reactions and improvement asks** (captured verbatim on the answer sheet) are the build backlog, collected by someone already standing in front of the user.
- **Connectivity notes from visits** answer OQ8 with data instead of guesses.

6. Open questions (the decision backlog this doc exists to hold)

#	Question	Why it matters / current lean
OQ1	**Retention policy per artifact class** ? original (only L3 has it), proxy, chunks, frames/trajectories, Gemini remote files, intervals/labels	intervals/labels are the asset (keep forever, tiny); originals/proxies need an explicit keep-while-active + delete-on-request rule; Gemini remote-file GC already in BACKLOG
OQ2	**Which file's MD5 is the video's identity** at L2/L1	lean: the processed file's MD5 (proxy), with original_md5 carried in metadata ? the collector manifest pattern, promoted to a product rule
OQ3	**Consent language for the Gemini path** ? even "local" modes ship 1280px chunks to Google	must be plain in the academy-facing terms; paid-tier no-training posture to be cited
OQ4	**Key custody for L1-direct** ? client calls Gemini with whose key? per-user metering, abuse, cost ceiling per video	unsolved; this is what separates L1 from L1-direct
OQ5	**Windows client packaging** ? portable ffmpeg + a signed tiny app vs a PowerShell script for the pilot	pilot can be a script the associate/owner runs; product needs

an installer story (ffmpeg LGPL build = licensing-clean) |
132 | OQ6 | **What does the review UI play** at each tier | L2 streams the proxy (good); L1 has only chunks ? review degrades ? this quietly argues L2 over L1 as the default |
133 | OQ7 | **Reel delivery** ? server-compiled from proxy (shareable 1080p) vs intervals-back + client cuts full-res | could be both: proxy reel by default, "premium full-res" cut locally |
134 | OQ8 | **Real academy uplink numbers** ? the L2-vs-L3 case rests on bandwidth assumptions | C13 visits can note connectivity informally; don't redesign around guesses |
135 | OQ9 | **Recurring volume** ? an academy recording daily generates 10 GB/day; even L2 accumulates | retention (OQ1) + maybe future delta thinking; not an alpha problem |
136 | OQ10 | **If L3 must survive** (users who refuse any client-side step) | needs resumable uploads + a paid edge tier + a raised cap; treat as the exception path, not the design center |
137
138   ## 7. Where this lands (recommendation, not a greenlight)
139
140   **Default the alpha to L2 (proxy upload):** a one-command client prep step built
141   on the *already-verified* proxy contract, uploads that fit comfortably inside
142   today's chunked endpoint and 4 GB cap, a GPU-less serving path (the affordability
143   requirement, §4, satisfied by construction), a review UI that streams the proxy,
144   and intervals that map 1:1 onto the customer's original.
145   **L1/L1-direct is the fast-follow** once OQ4 has an answer; **L0 is the owned-
146   model milestone**, not an alpha decision. The field pilot keeps running on L4 ?
147   a person with a hard drive is a fine storage backend for n=3 academies.
148

4. assistant (2026-06-19T19:17:38.328Z)

I have read the doc. It's a focused thinking doc on video locality. Let me check whether the companion sections it references add any BYO-cloud-storage tier, since the doc explicitly points to `PLATFORM_ARCHITECTURE.md` §3 (StorageBackend registry) as the home for storage backends.

5. user (2026-06-19T19:17:40.468Z)

15:browsable by history**, and choose **where their data lives** (Google Drive / cloud
16:bucket / local) and **where the compute runs** (our GPU / their VM). The organizing
41:> Spiideo offer true BYO-storage or BYO-compute ? they're vendor-cloud SaaS
58:| Storage | local `filepath` string; output to `output_dir` | no abstraction; can't BYO Drive/bucket; raw video must be on the box |
85:   PLANE   ? (local | BYO-VM | ? ? (per-tenant DB: ?
92:                ? Storage (local upload | Google Drive | S3 | ?
93:                ? GCS | Azure) ? raw video + highlight outputs ?
109:### 3.1 `StorageBackend` registry
114:class StorageBackend(ABC):
125: local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) + Google Drive via `gdrivefs`,
126: Dropbox via `dropboxdrivefs`), reaching for **`smart_open`** as the robust large-MP4
127: byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as
128: add-ons ? less battle-tested than `s3fs`/`gcsfs`; cover with integration tests.)
129:- **Backends:** `local` (upload dir), `gdrive`, `s3`, `gcs`, `azure`. **BYO-storage**:
130: the tenant registers their own bucket/Drive; raw video **never leaves their account**.
131:- **Auth ? never persist users' long-lived keys.** For buckets, have the user create an
132: IAM role you **`AssumeRole`** into, scoped to one bucket/prefix, and mint **presigned
133: URLs** so raw video moves **browser ? the user's bucket directly, never through our
134: servers**. For Drive, OAuth with the narrow **`drive.file`** scope; for academies, a
148:- **`byo_worker`** ? the tenant runs a **pull-based worker** (a small agent) on their own
155: credential-safe v1** for BYO-compute.
158: (BYOC: everything launches in the user's accounts/VPCs; creds stay with them).
165: treat `byo_worker`/SkyPilot as a **power-user / enterprise / privacy-sensitive tier**,
166: not the academy default. If/when BYO ships, plan for **auto-updating, self-healing
168:- **Middle tier to consider ? "zero-infra BYO":** a managed-but-isolated path where the
169: **storage** stays in the user's bucket (signed URLs) while compute runs on **our** GPUs
171: This gives the BYO *privacy* benefit (raw video never persists on our infra) **without**
172: the user running a worker ? a pragmatic default between full-hosted and full-BYO.
234:- **Dispatch:** the orchestrator enqueues; a worker (hosted or `byo_worker`) claims jobs
235: for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline,

309:inference jobs (even **BYO-compute** can serve inference on the tenant's GPU); the
356:| **ms-swift** (ModelScope) | Self-managed (rented/own GPU) | **Native video+audio**; Qwen3-VL/Omni, 300+ MLLMs | any incl **7B** | ? DPO/GRPO | ? full ownership | **? video-QA; slots into BYO-compute (§3.2)** |
372: **ComputeBackend / BYO-compute** seam (§3.2). **Likely a better fit than Tinker** for
401:**Engine (don't reinvent):** **`ms-swift`** already delivers this for VLMs ? CLI-driven,
443: **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's bucket
458:    your storage, your compute ? and now **your model**.* With BYO-compute the adapter can be
477:| **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry; `local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds |
478:| **P3** | **`ComputeBackend` registry + BYO-compute** | `local`/`hosted`/`byo_worker`; pull-based worker agent; control/data-plane split |
479:| **P4** | **Rich rally/shot semantic metadata & tagging** *(the next value stage after solid rally detection + highlights)* | shot type (smash / backhand drive / net), rally **outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ? query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden set via the `annotation` registry + `candidate_segments` |
486:that ended in a smash or backhand drive,"* *"rallies where the court was open but the
507:- **BYO-storage is the central privacy/liability lever:** if raw video stays in the
508: tenant's own bucket/Drive and we process via signed URL or their compute, we minimize
518: what merely passes through a tenant's bucket. **Exclude minors' video from the training
532:1. **Hosted-first vs BYO-first.** Recommended: **hosted control-plane first** (P0?P1) with
533:   **BYO-storage soon after** (P2); **BYO-compute later** (P3) as a power-user/enterprise
542:6. **How far to push BYO-compute** given its security/ops complexity.
552:tests); BYO-compute security; **biometric/minor liability** (get this right before any
564:[Omitted long matching line]
568:[Omitted long matching line]

6. assistant (2026-06-19T19:17:45.811Z)

The companion doc `PLATFORM_ARCHITECTURE.md` has extensive BYO-storage design (a `StorageBackend` registry with `gdrive`/`s3`/`gcs`/`azure`/dropbox backends). But the question is specifically about `VIDEO_LOCALITY_MODEL.md`. Let me verify whether the locality doc mentions a watch-folder/autosync notion at all, and check the §2 ladder precisely for whether "user's own cloud" is a distinct tier vs. a passing reference.

The doc is short and I've read it fully. Let me confirm there's no "watch"/"poll"/"sync"/"automatic"/"trigger on arrival" language in it.

7. user (2026-06-19T19:17:51.438Z)

7:`PLATFORM_ARCHITECTURE.md` §3 (future StorageBackend/ComputeBackend registries),
10:## 0. The questions that triggered this (and the short answers)
25:miserable everywhere; expecting academies to manage S3/GCS buckets is a
60:| **L4 ? sneakernet** | a hard drive | 0 over network | a person (the field associate / owner) | how the pilot actually works today |
99: disk); settle retention (OQ1) before volume arrives; the free edge tier stays
115: back on a drive (L4), we proxy + process + return a reel ? the same motion
116: L2 will automate. Every friction the associate hits there (file handoff,
135:| OQ9 | **Recurring volume** ? an academy recording daily generates 10 GB/day; even L2 accumulates | retention (OQ1) + maybe future delta thinking; not an alpha problem |
147:a person with a hard drive is a fine storage backend for n=3 academies.

8. assistant (2026-06-19T19:17:56.116Z)

I now have a complete, evidence-based picture. The `VIDEO_LOCALITY_MODEL.md` doc does NOT have a tier for "the user's own cloud storage" and never discusses autosync/watch-folder. Its entire ladder is framed around *what artifact crosses the network from the academy desktop to OUR box*, not around the video already living in the user's cloud. The only mention of S3/GCS buckets (line 25) is dismissive ("expecting academies to manage S3/GCS buckets is a non-starter"). The BYO-storage concept lives entirely in the companion `PLATFORM_ARCHITECTURE.md` §3, which this doc only points to.

9. user (2026-06-19T19:18:39.285Z)

Structured output provided successfully

10. assistant (2026-06-19T19:18:48.533Z)

I've assessed the idea against `docs/VIDEO_LOCALITY_MODEL.md` and returned the structured object.

Bottom line: the locality doc does NOT anticipate BYO-cloud-storage + autosync. Its entire L0?L4 ladder classifies designs by "smallest artifact that must cross OUR network *from the academy desktop*", not by "the original lives in the user's own cloud". The words watch/poll/sync/webhook never appear; the only S3/GCS mention (line 25) is dismissive. The actual home for this idea is the companion `docs/PLATFORM_ARCHITECTURE.md` §3.1 `StorageBackend` registry (which already designs `gdrive`/`s3`/`gcs`/`azure` backends, OAuth `drive.file` scopes, and presigned-URL custody) plus a new watch/sync trigger feeding the committed-next async job model. Conceptually it maps best as an "automated L4" ? the networked, unattended cousin of sneakernet ? and the doc's own §1 insight ("cut the reel wherever the original lives") is precisely what makes it viable, but that connection is not drawn here.